

第二讲 缓冲区溢出基础

一、缓冲区溢出概述

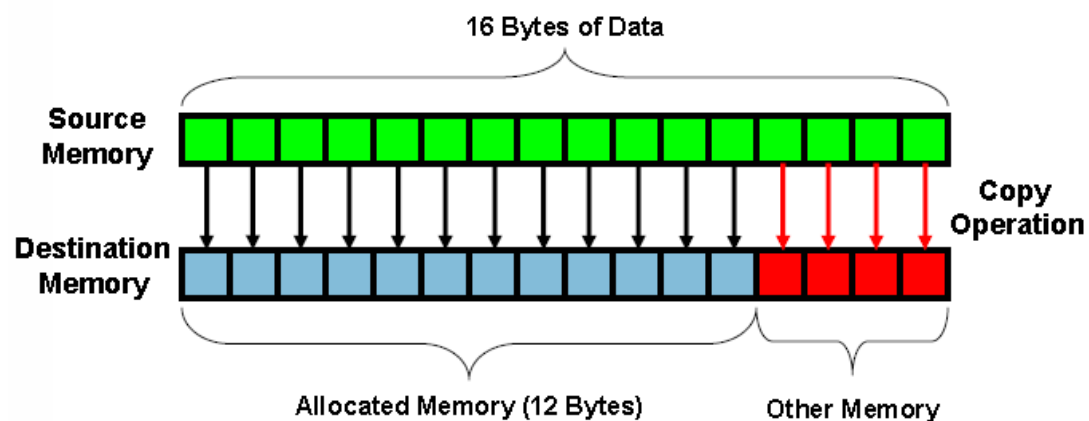
1、缓冲区溢出原理简介

缓冲区溢出是指当计算机向缓冲区内填充数据位数时超过了缓冲区本身的容量溢出的数据覆盖在合法数据上。

这种错误的状态发生在写入内存的数据超过了分配给缓冲区的大小的时候,就像一个杯子只能盛一定量的水,如果放到杯子中的水太多,多余的水就会一出到别的地方。由于缓冲区溢出,相邻的内存地址空间被覆盖,造成软件出错或崩溃。如果没有采取限制措施,可以使用精心设计的输入数据使缓冲区溢出,从而导致安全问题。

缓冲区溢出是最常见的内存错误之一,也是攻击者入侵系统时所用到的最强大、最经典的一类漏洞利用的方式。

缓冲区溢出图示:



2、缓冲区溢出问题历史

(1) 很长一段时间以来,缓冲区溢出都是一个众所周知的安全问题, C 程序的缓冲区溢出问题早在 70 年代初就被认为是 C 语言数

据完整性模型的一个可能的后果。这是因为在初始化、拷贝或移动数据时，C 语言并不自动地支持内在的数组边界检查。虽然这提高了语言的执行效率，但其带来的影响及后果却是深远和严重的。

- 1988 年 Robert T. Morris 的 finger 蠕虫程序。这种缓冲区溢出的问题使得 Internet 几乎限于停滞，许多系统管理员都将他们的网络断开，来处理所遇到的问题。

- 1989 年 Spafford 提交了一份关于运行在 VAX 机上的 BSD 版 UNIX 的 fingerd 的缓冲区溢出程序的技术细节的分析报告，引起了部分安全人士对这个研究领域的重视。

- 1996 年出现了真正有教育意义的第一篇文章，Aleph One 在 Underground 发表的论文详细描述了 Linux 系统中栈的结构和如何利用基于栈的缓冲区溢出。

(2) Aleph One 的贡献还在于给出了如何写开一个 shell 的 Exploit 的方法，并给这段代码赋予 shellcode 的名称，而这个称呼沿用至今，我们现在对这样的方法耳熟能详--编译一段使用系统调用的简单的 C 程序，通过调试器抽取汇编代码，并根据需要修改这段汇编代码。

- 1997 年 Smith 综合以前的文章，提供了如何在各种 Unix 变种中写缓冲区溢出 Exploit 更详细的指导原则。

- 1998 年 来自 “Cult of the Dead Cow” 的 Dildog 在 Bugtrq 邮件列表中以 Microsoft Netmeeting 为例子详细介绍了如何利用 Windows 的溢出，这篇文章最大的贡献在于提出了利用栈指针的方法来完成

跳转，返回地址固定地指向地址，将 Windows 下的溢出 Exploit 推进了实质性的一步。

3、缓冲区溢出的影响

不要小看缓冲区溢出问题,它可能会产生很恶劣的影响:

- 发布安全报告以及相关补丁的时间和费用开销
- 数以千计的系统管理员安装补丁的时间开销
- 系统被攻击者破坏所造成的损失
- 重要资料被盗取造成的损失
- 开发者的信誉...

粗略的算下来，一个马虎的错误就可能需要花费几百万美元的代价才能弥补。

4、缓冲区溢出的预防

面临越来越多的来自缓冲区溢出攻击的威胁，例如 Immunix 根据自动侦测和预防技术开发的 StackGuard 系统之类的侦测和防止缓冲区溢出发生的自适应技术被研发出来.防范溢出问题正受到越来越多的关注。

目前对于缓冲区溢出，主要分为静态保护和动态保护：

● **静态保护**：不执行代码,通过静态分析来发现代码中可能存在的漏洞。静态的保护技术包括编译时加入限制条件，返回地址保护，二进制改写技术，基于源码的代码审计等。

● **动态保护**：通过执行代码分析程序的特性，测试是否存在漏洞，或者是保护主机上运行的程序来防止来自外部的缓冲区溢出攻击。

二、系统栈的工作原理

1、PE 文件格式简介

(1) PE (Portable Executable) 是 Win32 平台下可执行文件遵守的数据格式。常见的可执行文件(如 “*.exe” 文件和 “*.dll” 文件)都是典型的 PE 文件。

(2) 一个可执行文件不光包含了二进制的机器代码，还会包含许多其他信息，如字符串、菜单、图标、位图、字体等。

(3) PE 文件格式规定了所有的这些信息在可执行文件中如何组织。在程序被执行时，操作系统会按照 PE 文件格式的约定去相应的地方准确地定位各种类型的资源，并分别装入内存的不同区域。

(4) PE 文件格式把可执行文件分成若干个数据节(section)，不同的资源被存放在不同的节中。一个典型的 PE 文件包含的节如下：

→ .text 由编译器产生，存放着二进制的机器代码，也是我们反汇编和调试的对象。

→ .data 初始化的数据块，如宏定义、全局变量、静态变量等。

→ .idata 可执行文件所使用的动态链接库等外来函数与文件的信息。

→ .rsrc 存放程序的资源，如图标、菜单等。

→ 除此之外，还可能出现的节包括 “.reloc ”、“.edata”、“.tls”、“.rdata” 等。

(5) PE 文件简单构成

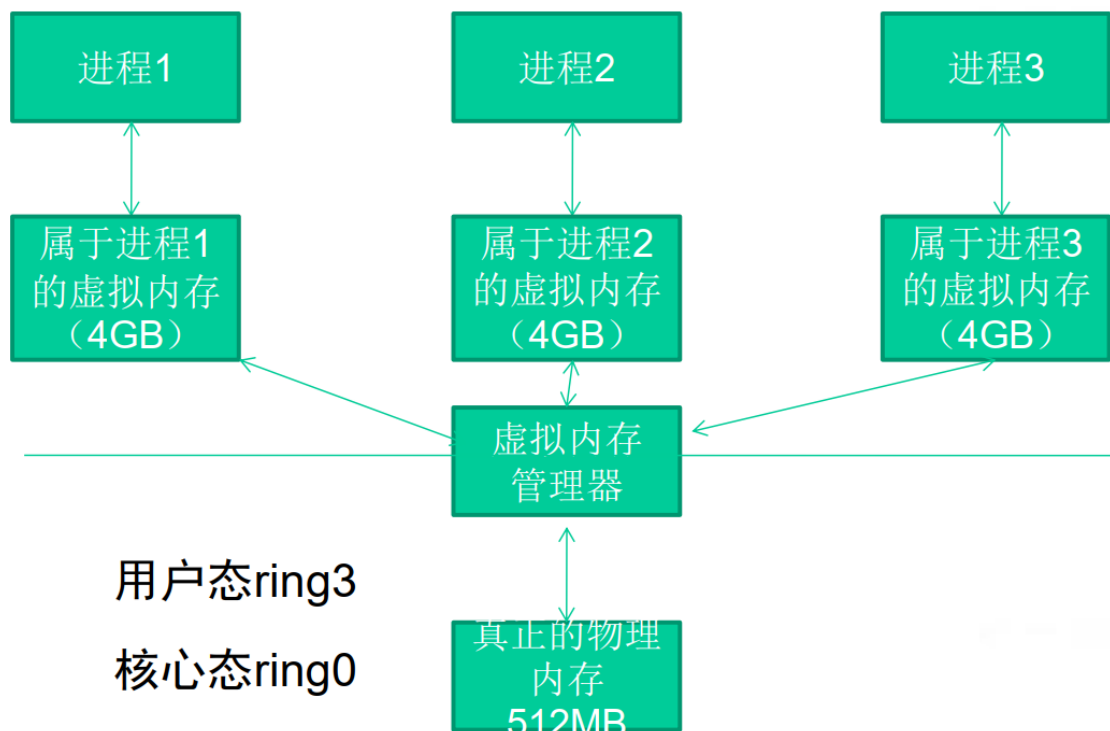


2、虚拟内存相关知识

(1) 虚拟内存简介

Windows 的内存可以被分为两个层面：物理内存和虚拟内存。其中，物理内存比较复杂，需要进入 Windows 内核级别 ring0 才能看到。通常，在用户模式下，我们用调试器看到的地址都是虚拟内存。

Windows 让所有的进程都“相信”自己拥有独立的 4GB 内存空间。但是我们计算机中那跟实际的内存条可能只有 512MB，怎么能为所有进程都分配 4GB 的内存呢？这一切都是通过虚拟内存管理器的映射做到的。



虽然每个进程都“相信”自己拥有 4GB 的空间，但实际上它们运行时真正能用到的空间根本没有那么多。

内存管理器只是分给进程一片“假地址”，或者说是“虚拟地址”，它们对进程来说只是一笔“无形的数字财富”；

当需要实际的内存操作时，内存管理器才会把“虚拟地址”和“物理地址”联系起来。

(2) 内存管理与银行的类比

内存管理	银行类比
进程	储户
内存管理器	银行
物理内存	钞票
虚拟内存	存款
进程可能拥有大片内存，但使用往往很少	储户拥有大笔存款，但实际生活中的开销并没多少

内存管理	银行类比
进程不使用虚拟内存时，这些内存只是些地址，是虚拟存在的，是一笔无形的数字财富。	用户不使用储蓄时，储蓄也只是一些数字，是无形的数字财富。
进程使用内存时，内存管理器会为器会为这个虚拟地址映射实际的物理地址，虚拟内存地址和最终被映射到的物理内存地址之间没有什么必然联系	储户需要钱时，银行才会兑换一定的现金给储户，但物理钞票的号码与储户心目中的数字存款之间可能并没有任何联系。

内存管理	银行类比
操作系统的实际物理内存空间可以远远小于进程的虚拟内存空间之和，仍能正常调度	银行的现金准备可以远远小于所有储户的储蓄额总和，仍能正常运转
很少出现所有程序要申请出全部物理内存	很少会出现所有储户同时要取出全部存款的现象
物理内存可以远小于虚拟内存之和	社会上实际流通的钞票可以远远小于社会的财富总额

(3) PE 文件与虚拟内存之间的映射

静态反汇编工具看到的 PE 文件中某条指令的位置是相对于磁盘文件而言的，即所谓的文件偏移，我们可能还需要知道这条指令在内存中所处的位置，即虚拟内存的位置。

反之，在调试时看到的某条指令的地址是虚拟内存地址，我们也经常需要回到 PE 文件中找到这条指令对应的机器码。

几个概念：

- 文件偏移地址(File Offset)

➤ 数据在 PE 文件中的地址叫做文件偏移地址。这是文件在磁盘上存放时相对于文件开头的偏移。

● 装载基址(Image Base)

➤ PE 装入内存时的基地址。默认情况下, EXE 文件在内存中的基地址是 0x00400000, DLL 文件是 0x10000000。这些位置可以通过修改编译选项更改。

● 虚拟内存地址(Virtual Address, VA)

➤ PE 文件中的指令被装入内存后的地址。

● 相对虚拟地址(Relative Virtual Address, RVA)

➤ 相对虚拟地址是内存地址相对于映射基址的偏移量。

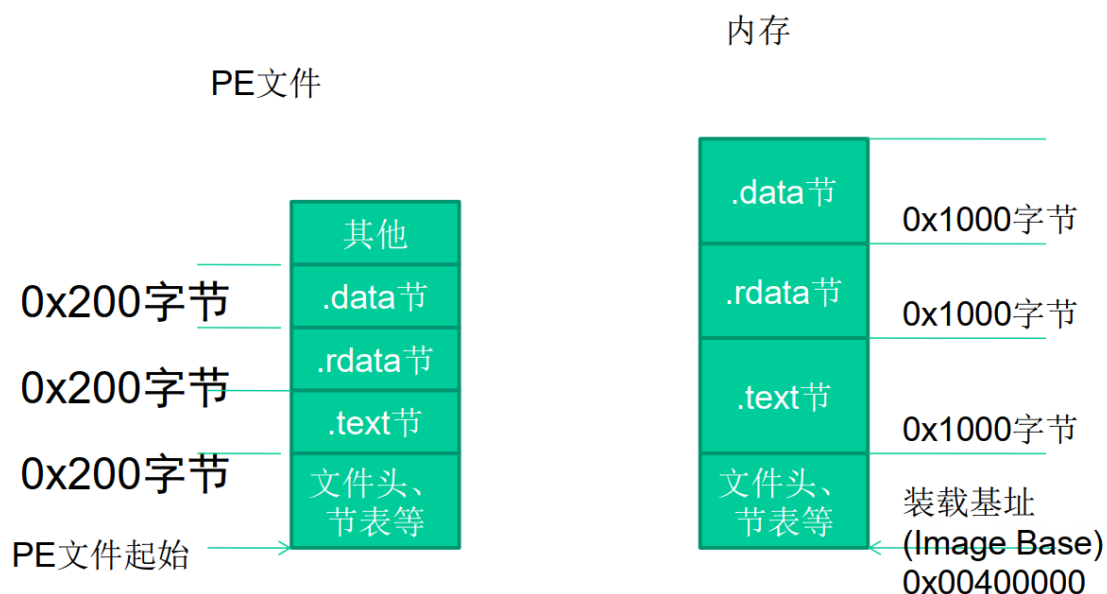
➔ 虚拟内存地址、映射基址、相对虚拟内存地址三者有如下关系 $VA = Image\ Base + RVA$ 。

文件偏移地址在与他们计算时还需要考虑存放方式的不同。

● PE 文件中的数据按照磁盘数据标准存放, 以 0x200 字节为基本单位进行组织。当一个数据节不足 0x200 字节时, 不足的地方将被 0x00 填充; 当一个数据节超过 0x200 字节时, 下一个 0x200 块将分配给这个节使用。因此 PE 数据节的大小永远是 0x200 的整数倍。

● 当代码装入内存后, 将按照内存数据标准存放, 并以 0x1000 字节为基本单位进行组织。类似的, 不足将被不全, 若超出将分配下一个 0x1000 为其所用。因此, 内存中的节总是 0x1000 的整数倍。

PE 文件与虚拟内存的映射关系：



我们把这种由存储单位差异引起的节基址差称做节偏移，在下图例中：

$$.text \text{ 节偏移} = 0x1000 - 0x400 = 0xc00$$

$$.rdata \text{ 节偏移} = 0x7000 - 0x6200 = 0xE00$$

$$.data \text{ 节偏移} = 0x9000 - 0x7400 = 0x1c00$$

$$.rsrc \text{ 节偏移} = 0x2D000 - 0x7800 = 0x25800$$

文件偏移地址

$$= \text{虚拟内存地址(VA)} - \text{装载基址(Image Base)} - \text{节偏移}$$

$$= \text{RVA} - \text{节偏移}$$

以下表为例，如果在调试时遇到虚拟内存中 0x00404141 处的一条指令，那么要换算出这条指令在文件中的偏移量，有：

$$\begin{aligned} \text{文件偏移量} &= 0x00404141 - 0x00400000 - (0x1000 - 0x400) \\ &= 0x3541 \end{aligned}$$

节(section)	相对虚拟偏移量(RVA)	文件偏移量
.text	0x00001000	0x0400
.rdata	0x00007000	0x6200
.data	0x00009000	0x7400
.rsrc	0x0002D000	0x7800

3、内存的不同用途

成功地利用缓冲区溢出漏洞可以修改内存中的变量的值，甚至可以劫持进程，执行恶意代码，最终获得主机的控制权。要透彻地理解这种攻击方式，我们需要回顾一些计算机体系架构的基础知识，搞清楚 CPU、寄存器、内存是怎样协同工作而让程序流畅执行的。

(1) 寄存器

Intel x86 的寄存器可以分为下述的几类：

● 通用寄存器

→ 32 位的通用寄存器有 EAX、EBX、ECX、EDX、ESP、BP、ESI 和 EDI，它们的使用方法不总是相同的。一些指令赋予它们特殊的功能。

→ 在通用寄存器里面有很多寄存器虽然他们的功能和使用没有任何的区别，但是在长期的编程和使用中，在程序员习惯中已经默认的给每个寄存器赋上了特殊的含义，比如：

EAX 一般用来做返回值

ECX 用于计数

EIP：扩展指令指针。在调用一个函数时，这个指针被存储在堆栈中，用于后面的使用。在函数返回时，这个被存储的地址被用于决定下一个将被执行的指令的地址。

ESP：扩展堆栈指针。这个寄存器指向堆栈的当前位置，并允许通过使用 push 和 pop 操作或者直接的指针操作来对堆栈中的内容进行添加和移除。

EBP：扩展基指针。主要用与存放在进入 call 以后的 ESP 的值，便于退出的时候回复 ESP 的值，达到堆栈平衡的目的。

● 段寄存器

→ 段寄存器被用于指向进程地址空间不同的段。

→ CS 指向一个代码段的开始；

→ SS 是一个堆栈段；

→ DS、ES、FS、GS 和各种其他数据段，例如存储静态数据的段。

● 程序流控制寄存器

● 其他寄存器

根据不同的操作系统，一个进程可能被分配到不同的内存区域去执行。但是不管什么样的操作系统、什么样的计算机架构，进程使用的内存都可以按照功能大致分成以下 4 个部分。

名称	用途
代码区	这个区域存储着被装入执行的二进制机器代码，处理器会到这个区域取指并执行。
数据区	用于存储全局变量等。
堆区	进程可以在堆区动态地请求一定大小的内存，并在用完之后归还给堆区。动态分配和回收是堆区的特点。
栈区	用于动态地存储函数之间的调用关系，以保证被调用函数在返回时恢复到父函数中继续执行。

如果把计算机看成一个有条不紊的工厂，我们可以得到如下类比。

名称	类比
CPU	完成工作的工人
数据区、堆区、栈区等	存放原料、半成品、成品等各东西的场所。
存在代码区的指令	告诉CPU要做什么，怎么做，到哪里去领原料，用什么工具来做，做完以后把成品放到哪个货舱去。
栈区	栈除了扮演存放原料、半成品的仓库之外，它还是车间调度主任的办公室。

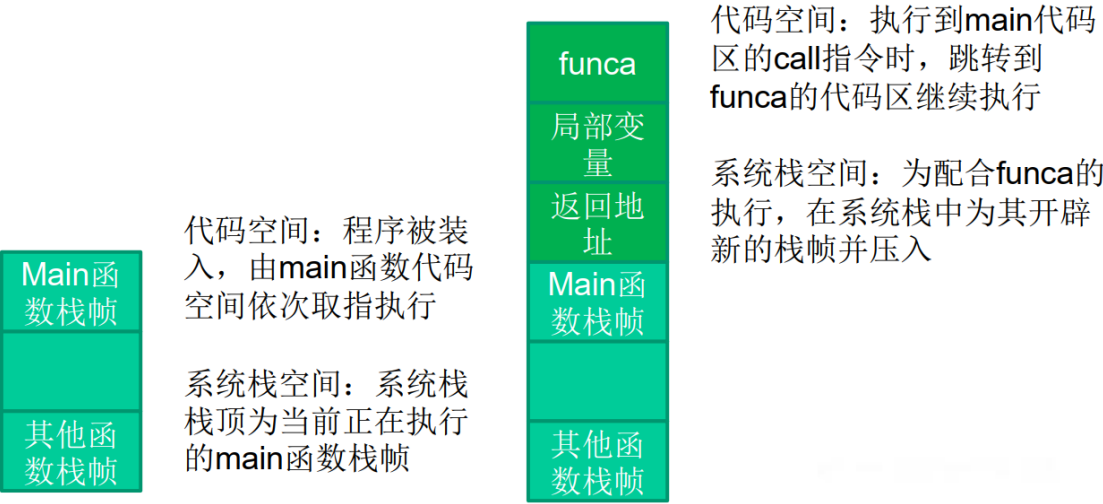
4、栈与系统栈

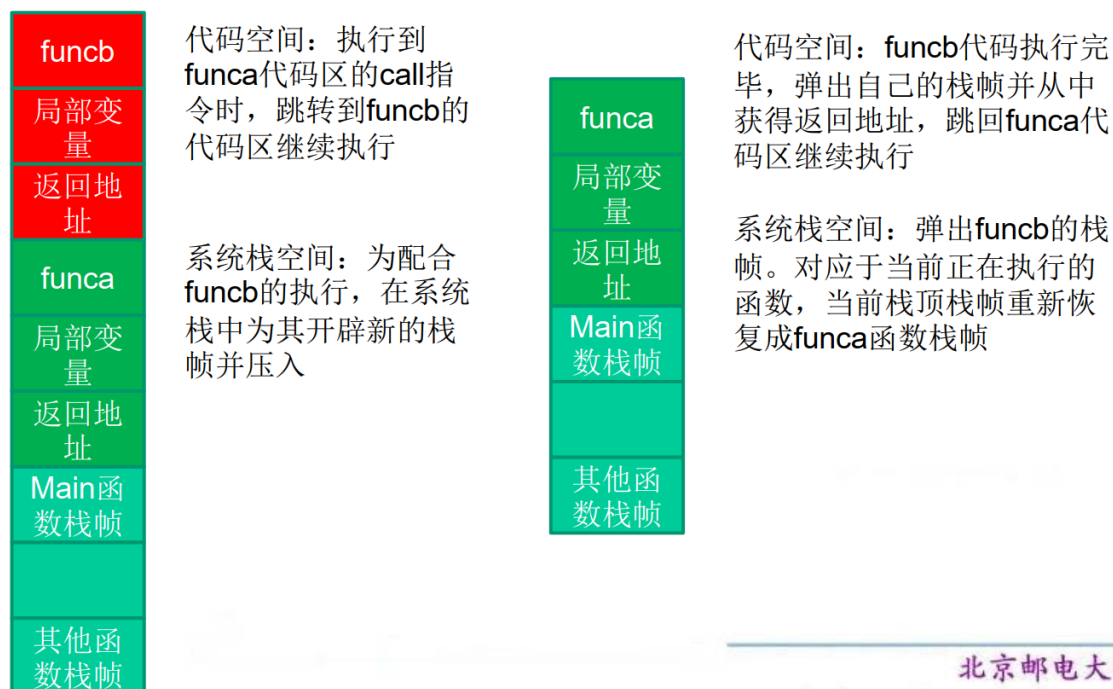
从计算机科学的角度来看，栈指的是一种数据结构，是一种先进后出的数据表。

栈的最常见操作有两种：压栈(PUSH)、弹栈(POP)；用于标识栈的属性也有两个：栈顶(TOP)、栈底(BASE)。

内存的栈区实际上指的就是系统栈。系统栈由系统自动维护，它用于实现高级语言中函数的调用。对于类似 C 语言这样的高级语言，系统栈的 PUSH/POP 等堆栈平衡细节是透明的。

5、函数调用时发生了什么





代码空间：**funca** 代码执行完毕，弹出自己的栈帧并从中获得返回地址，跳回 **main** 代码区继续执行

系统栈空间：弹出 **funca** 的栈帧。对应于当前正在执行的函数，当前栈顶栈帧重新恢复成 **main** 函数栈帧

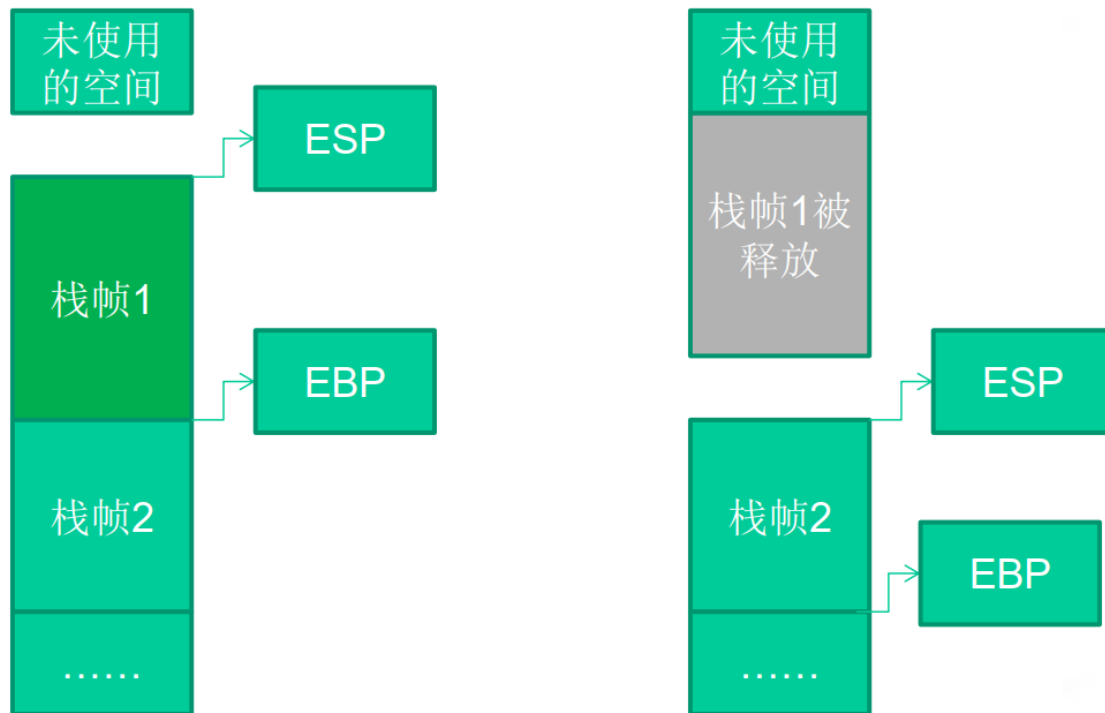
6、寄存器与函数栈帧

每一个函数独占自己的栈帧空间。当前正在运行的函数的栈帧总是在栈顶。Win32 系统提供两个特殊的寄存器用于标识位于系统栈顶端的栈帧。

(1) ESP：栈指针寄存器(extended stack pointer)，其内存放着一个指针，该指针永远指向系统栈最上面的一个栈帧的栈顶。

(2) EBP：基址指针寄存器(extended base pointer)，其内存放着一个指针，该指针永远指向系统栈最上面的一个栈帧的底部。

栈帧寄存器 ESP 与 EBP 的作用：



在函数栈帧中，一般包含以下几类重要信息。

(1) 局部变量：为函数局部变量开辟的内存空间。

(2) 栈帧状态值：保存前栈帧的顶部和底部（实际上只保存前栈帧的底部，前栈帧的顶部可以通过堆栈平衡计算得到），用于在本帧被弹出后恢复出上一个栈帧。

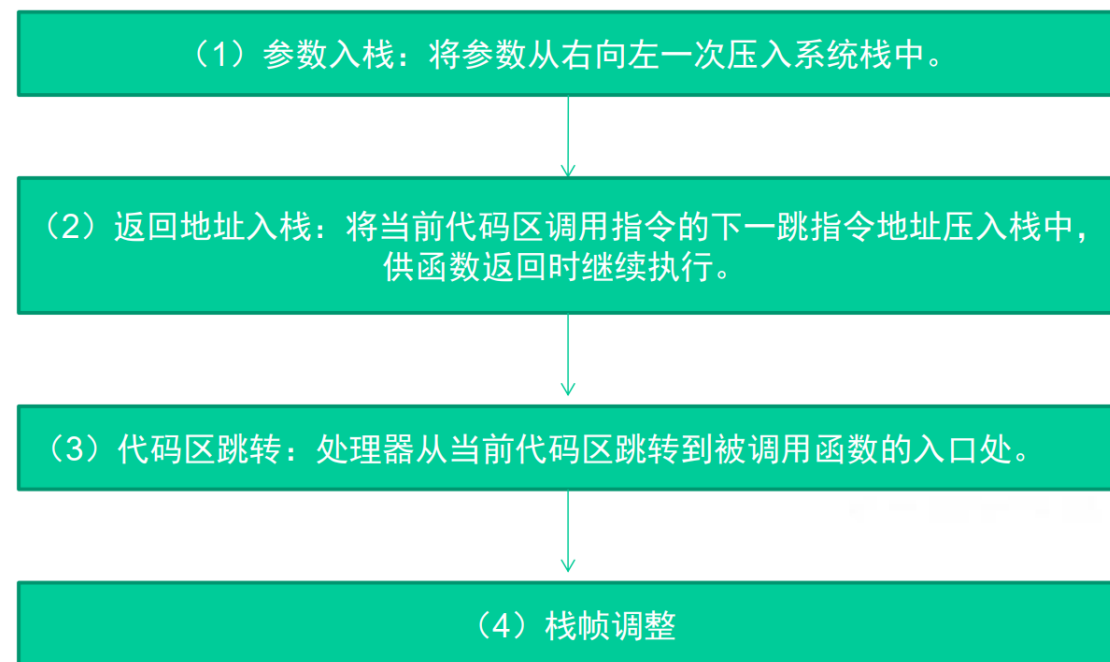
(3) 函数返回地址：保存当前函数调用前的“断点”信息，也就是函数调用前的指令位置，以便在函数返回时能够恢复到函数被调用前的代码区中继续执行指令。

EIP：指令寄存器 (Extended Instruction Pointer)，其内存放着一个指针，该指针永远指向一条等待执行的指令地址。

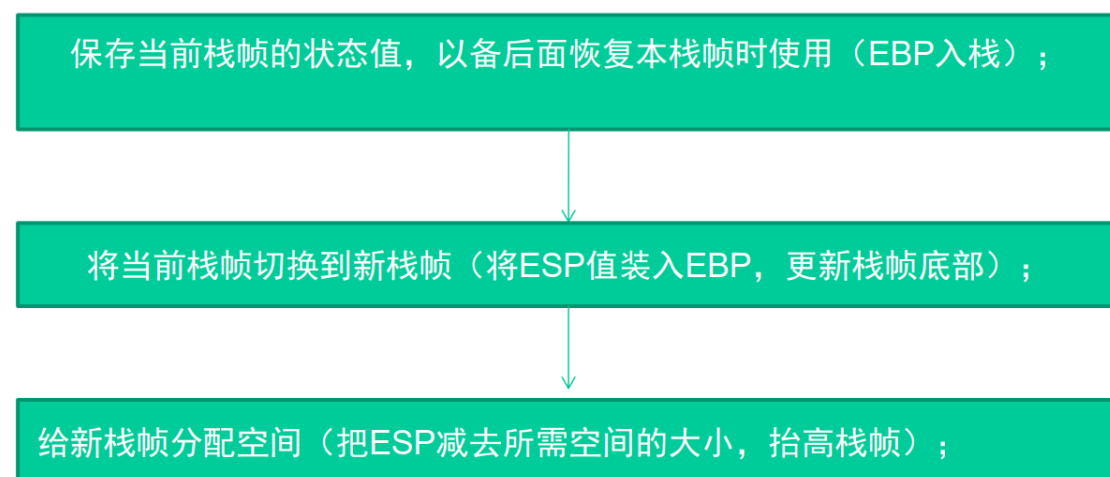
可以说如果控制了 EIP 寄存器的内容，就控制了进程---我们让 EIP 指向哪里，CPU 就会去执行哪里的指令。

7、函数调用约定与相关指令

函数调用大致包括以下几个步骤。



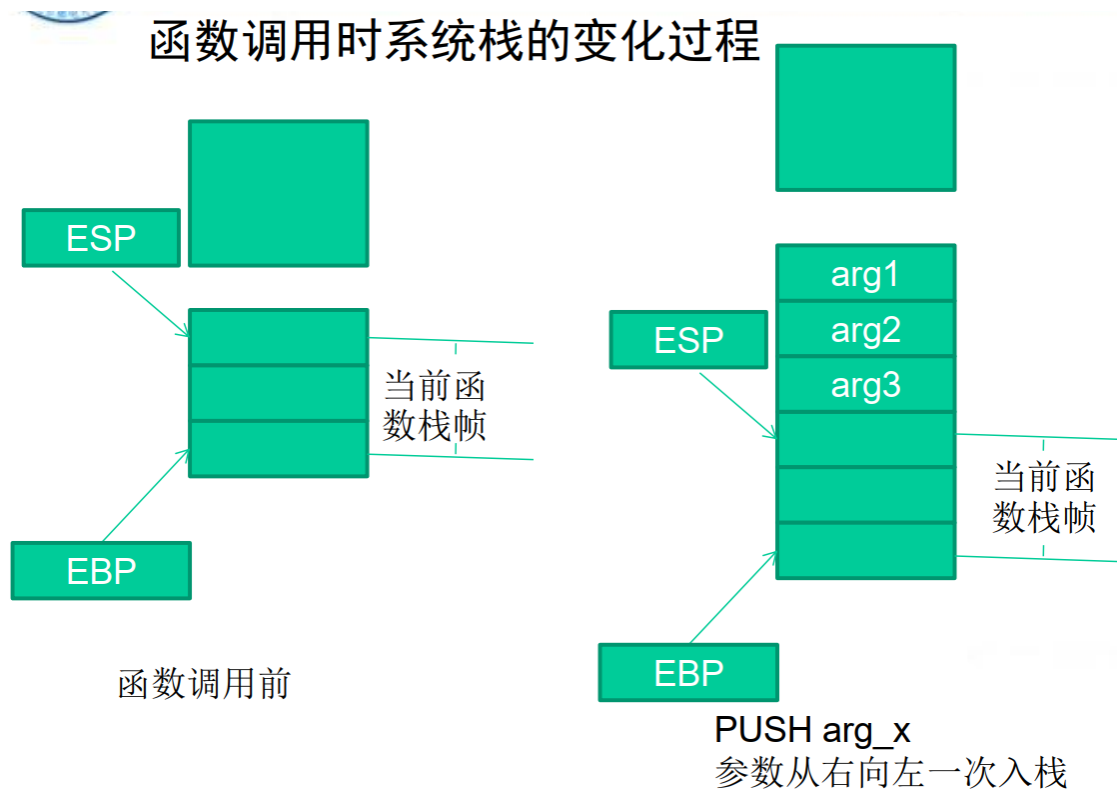
第四步栈帧调整具体包括如下几个步骤。

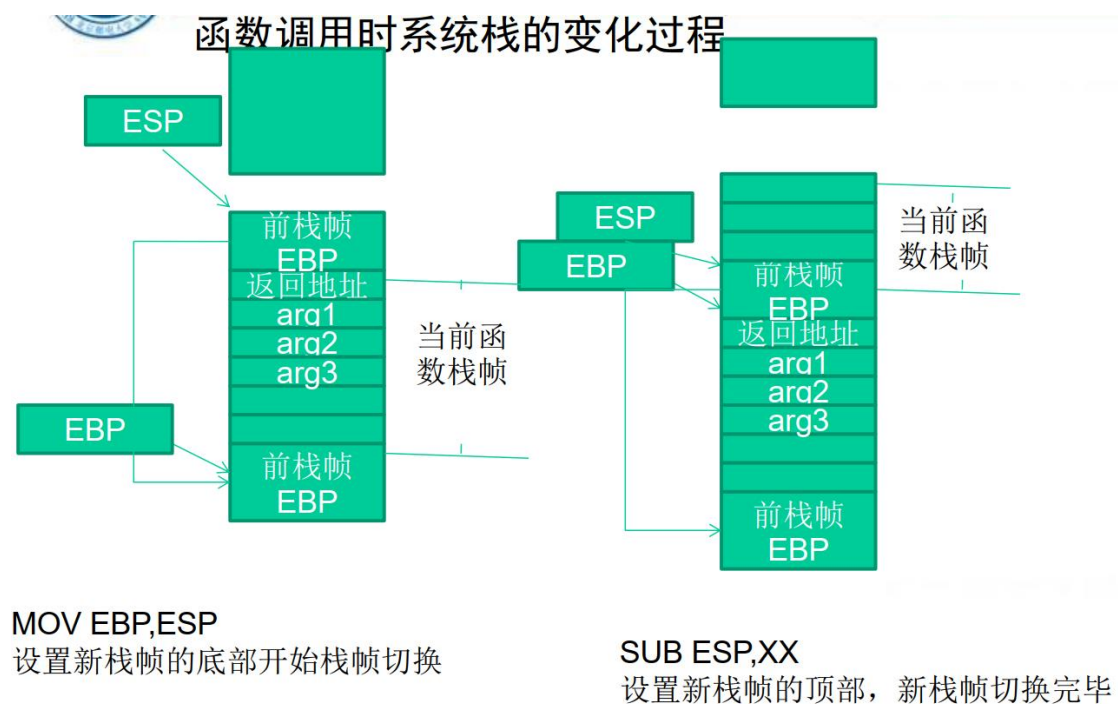
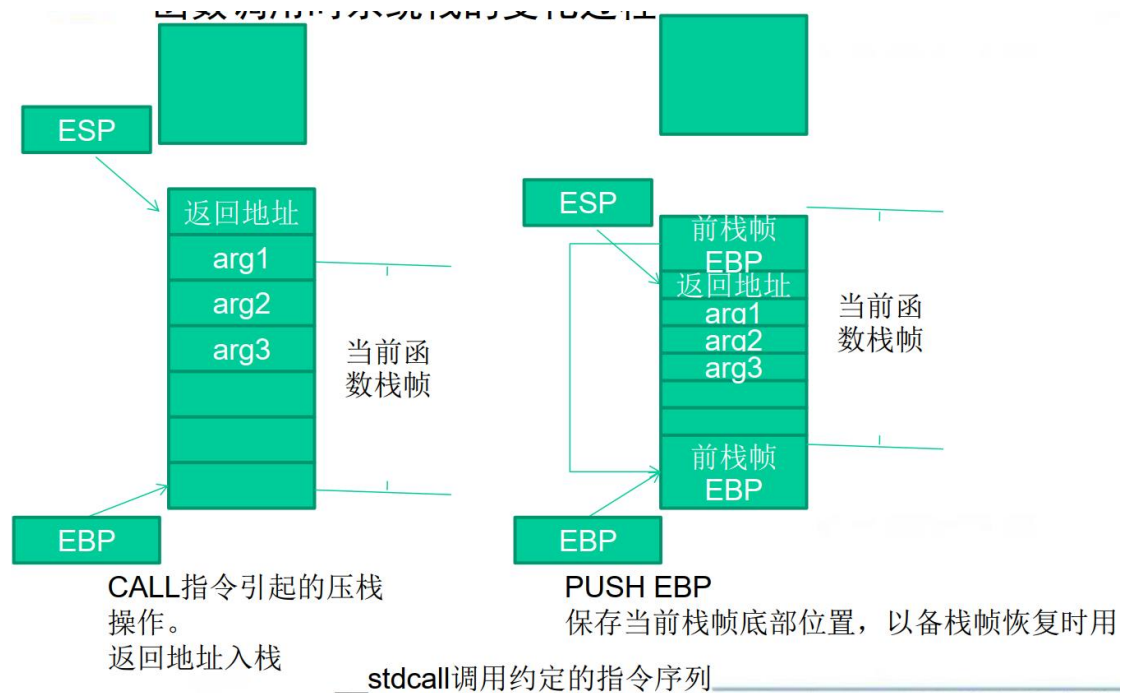


对于__stdcall 调用约定，函数调用时用到的指令序列大致如下。

序列	备注
push 参数 3	假设该函数有3个参数，将从右向左依次入栈
push 参数 2	
push 参数 1	
call 函数地址	call指令将同时完成两项工作:a)向栈中压入当前指令在内存中的位置，即保存返回地址。 b)跳转到所调用函数的入口地址函数入口处
push ebp	保存旧栈帧的底部
mov ebp,esp	设置新栈帧的底部(栈帧切换)
sub esp,xxx	设置新栈帧的顶部(抬高栈顶，为新栈帧开辟空间)

函数调用时系统栈的变化过程：





类似的，函数返回的步骤如下。

- (1) 保存返回值：通常将函数的返回值保存在寄存器 EAX 中。
- (2) 弹出当前栈帧，恢复上一个栈帧，具体包括：在堆栈平衡的基础上 给 ESP 加上栈帧的大小，降低栈顶，回收当前栈帧的空

间。将当前栈帧底部保存的前栈帧 EBP 值弹入 EBP 寄存器，恢复上一个栈帧。将函数返回地址弹给 EIP 寄存器。

(3) 跳转：按照函数返回地址跳回母函数中继续执行。

以 C 语言和 Win32 平台为例，函数返回时的相关的指令序列如下。

指令	备注
add esp,xxx	降低栈顶，回收当前的栈帧
pop ebp	将上一个栈帧底部位置恢复到ebp
retn	这条指令有两个功能： a)弹出当前栈顶元素，即弹出栈帧中的返回地址。至此，栈帧恢复工作完成。 b)让处理器跳转到弹出的返回地址，恢复调用前的代码区

三、堆栈溢出实例分析：修改邻接变量

如果这些局部变量中有数组之类的缓冲区，并且程序中存在数组越界的缺陷，那么越界的数组元素就有可能破坏栈中相邻变量的值，甚至破坏栈帧中所保存的 EBP 值、返回地址等重要数据。